Introdução:

Para a criação deste compilador básico de Pascal, decidimos escolher a linguagem Haskell. Apesar de C ter mais controlo na memória e na implementação, achamos que Haskell seria mais simples de usar e funciona bem para a criação da Abstract Syntax Tree. Após selecionarmos a linguagem, fomos ver exemplos de códigos em Pascal (e vimos um pouco da Wikipédia) para perceber a linguagem (semelhante a semântica). Algumas conclusões a que chegamos: é uma linguagem estruturada (código executa de forma linear), temos de ter cuidado com as suas palavras reservadas (program, var, begin, end, ...), bem como as expressões e instruções, respeitar ordem de operadores (* -> + -> <= -> else -> then), respeitar estrutura de blocos (begin/end, if/then/else). Após ver os exercícios da prática 2 como exemplo, passamos aos próximos passos:

Criação do Scanner (Lexer.x):

Criamos um Lexer inicial simples usando Alex. O Lexer servirá como analizador lexical do código fornecido, ou seja, lê o código Pascal todo e transforma numa sequência de tokens, respeitando as regras de precedência e associatividade de Pascal. Inicialmente, o nosso Lexer tinha as seguintes funções:

- definia o que eram algarismos, nºs, letras, identificadores e strings
- lia os nºs e ignorava comentários
- reconhecia alguns operadores aritméticos(+,-,*,/), parênteses, identificadores e outros símbolos matemáticos.
- criava tokens para os tais operadores/símbolos acima referidos
- criava tokens separados para os identifiers específicos do Pascal (como program, var e begin)

Criação do Parser.y:

Criamos um Parser inicial também simples usando Happy. O parser serve de analisador sintático, ou seja, recebe a lista de tokens do Lexer e substitui a gramática de expressões por Construtores AST(falamos mais à frente). Inicialmente, o Parser tinha as seguintes funções:

- identificava os tokens reconhecidos por ele próprio (iguais ao Lexer)
- lidava com precedência e associatividade dos operadores que já tínhamos(por exemplo, * tem maior precedência que +)
- criava uma gramática das expressões aritméticas, com regras para calcular contas matemáticas e como colocar parênteses

Criação da AST.hs:

Criamos a AST (Abstract Syntax Tree) inicial para termos alguns construtores de dados para cada tipo de nós(que o Parser chama). A AST cria esses construtores dentro de alguns tipos separados que nós criamos, como:

- PascalType(para Integer, Real, Bool, Char, String)
- Stmt(para o assign, write e read)
- WriteArg para dar print dos argumentos
- Expr para as expressões
- BinOp para contas matemáticas(Add, Multiply...), as boolean expressions colocamos mais tarde

Criação do Main.hs:

Criamos um Main.hs que recebe um código de exemplo em Pascal(neste caso um ficheiro txt como Example1.txt) e depois chama o Lexer e o Parser para compilar o código e imprimir os construtores da AST.

Criação do Example1.txt e Example2.txt:

Exemplos simples de códigos que tiramos da Internet(e alteramos), para testar o que já tínhamos feito com o Main.hs.

Alterações no Print:

Para imprimir a árvore AST, inicialmente dávamos print numa linha contínua, mas depois criamos o ASTPrinter.hs para tratar de dar print na árvore por níveis, com Tabs dependendo dos filhos de cada nó, dando-nos uma estrutura mais legível.

Adição dos If/Then/Else e While:

Após testarmos o Compilador com estas primeiras funcionalidades(contas matemáticas, writeln e readln), acrescentamos também a capacidade de interpretar os If e While:

- Acrescentamos Tokens no Lexer.x
- Pusemos regras de precedência dos "else", "then" e "do" e alteramos o Stmt no Parser.y
- Alteramos a data Stmt do AST.hs
- Acrescentamos os If e While no ASTPrinter para ele imprimir o conteúdo corretamente

Adição das expressões Booleanas:

Mais tarde acrescentamos funcionalidade para expressões booleanas(And, Or e Not), fazendo as seguintes alterações:

- Criação dos Tokens no Lexer.x e Expressões no Parser.y (e regras de precedência)
- Acrescentamos construtores na AST.hs para booleans(And, Or, Not)
- Mudamos o ASTPrinter.hs e o Semantics.hs para também usar UnaryExpr/UnOp (para Not) e BooleanExpr (para And e Or).

Alterações no Main.hs:

Agora o Main.hs usa o getContents em vez de readFile para podermos escrever no terminal o código logo ao testar o Compilador, em vez de dar feed de um script já feito.

Adição do ErrorCheck.hs:

Adicionamos um código extra que deteta variáveis não declaradas no programa Pascal e dá um erro personalizado a justificar. Também confirma nomes usados em destino de atribuições, mas não confirma tipos.

Testes finais:

Testamos vários códigos diferentes para ver alguns casos mais complicados, como ifs dentro de ifs, dois whiles seguidos, not(a and b) e alguns exemplos com assigns.

Instruções para usar código:

```
//para testar só o Lexer (com o Alex instalado), imprime os tokens:
alex Lexer.x -o Lexer.hs && ghci Lexer.hs
input <- readFile "Example1.txt"
mapM_ print (alexScanTokens input)

//para testar Lexer e Parser sem o Main(com o Alex/Happy instalados), imprime AST:
alex Lexer.x -o Lexer.hs && happy Parser.y -o Parser.hs && ghci -ignore-dot-ghci Lexer.hs
AST.hs Parser.hs -e 'readFile "Example1.txt" >>= print . Parser.parse .
Lexer.alexScanTokens'

//para testar Main com Example1, imprime tokens e AST:
alex Lexer.x -o Lexer.hs && happy Parser.y -o Parser.hs && ghc Main.hs -o teste && ./teste
< Example1.txt
```

//para testar Main com código escrito no Terminal:
alex Lexer.x -o Lexer.hs && happy Parser.y -o Parser.hs && ghc Main.hs -o teste
./teste
(escrever código aqui e terminar com CTRL+D)